Phase 1 | Master Project Overview

Understand the product, its architecture, and how to explain it.

FastAPI Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. Project in plain English

MetroInsights is a web planning prototype for exploring estimated hourly passenger demand at a selected Delhi Metro station and date. It combines a trained regression model, calendar context, service-operation assumptions, timetable/capacity calculations, and a line-aware route graph. The user selects a station, date, time, and optionally a corridor; the dashboard presents an estimate, a daily curve, a service recommendation, and shortest-path travel-time estimates.

A concise way to frame it: “I built a planning tool that turns station, date, and time inputs into a synthetic-data-based demand scenario, then compares that scenario with configurable service assumptions and gives network travel-time estimates.”

2. Problem and intended users

The product explores how a planner might compare demand scenarios across places and times. Its intended audience is a transit service planner or analyst, not a passenger seeking authoritative live travel advice. The interface helps make assumptions visible: forecast curve, estimated capacity utilization, suggested frequency/headway, date context, and route estimates. Since the underlying targets are synthetic, it demonstrates a workflow and model architecture rather than an operational decision system.

3. Main workflows

- User opens `/dashboard` and chooses a station/date/time/line.
- Browser calls `/api/predict` for dynamic dashboard data or submits `/get_footfall` for an HTML result.
- API parses inputs, determines whether service is operating, obtains calendar context, and invokes the forecasting logic.
- The forecaster builds the trained model's feature row from date/time and station metadata, then predicts one hourly value; minute-level display interpolates neighboring hourly estimates.
- The full-day profile predicts all 19 operating-hour slots, identifies the highest slot, sums the modeled daily total, and splits interchange demand using configured proportions.
- Scheduling logic compares line-allocated demand with modeled boarding clearance and constraints; it returns frequency/headway and risk labels.
- A graph search returns approximate shortest travel times to reachable modeled stations.
- Dashboard renders metrics, curves, recommendation text, and destinations.

4. Architecture map

Browser (HTML/CSS/JavaScript) -> FastAPI routes -> domain modules: forecasting, calendar context, operating hours, scheduling optimizer, topology/travel graph -> local files: model joblib, metadata joblib, station/operations CSV, topology/config JSON -> JSON or Jinja2 HTML response.

Startup sequence: Uvicorn imports `run:app`; `run.py` imports the `app` package; `app/__init__.py` constructs the FastAPI application and mounts static content; route module import creates templates, reads station metadata, and builds the travel graph; forecasting and scheduling modules load fitted artifacts and configuration. That eager work gives each worker

ready-to-use in-memory data but means missing/corrupt files can prevent startup and each worker has its own copy.

5. Model and data flow

The offline pipeline audits/cleans inputs, creates calibrated synthetic data, then engineers 23 columns and trains/evaluates models. Training saves the fitted estimator and inference metadata. At runtime, raw request values do not go directly to `predict`: they are transformed into the exact feature names listed in metadata. Station index mappings, historical aggregates, calendar features, line/typology indicators, and cyclic encodings must match training.

6. Network mapping: the idea in detail

Mapping is the translation between several representations that must agree:

- **Canonical station record:** the cleaned station table provides the display name, line metadata, interchange flag, and coordinates.
- **Topology label:** ordered route arrays use names to describe a route.
- **Alias/normalization mapping:** normalization removes marker characters and smooths punctuation/case differences; explicit aliases resolve remaining spellings. The result maps a route label to a canonical station name.
- **Graph state:** a physical interchange is expanded into one node per line, e.g. `(Rajiv Chowk, Yellow Line)` and `(Rajiv Chowk, Blue Line)`. This avoids an implicit zero-cost transfer.
- **Edges:** consecutive stops in each ordered line segment connect in both directions. Only explicitly configured interchange pairs connect the line-specific nodes, at a transfer penalty.

Why line-expanded nodes matter: if the graph used only one node called Rajiv Chowk, a route could switch lines without time or a transfer. If it used separate line-nodes with no connector, interchanges would be impossible. Explicit connector edges model a transfer cost and make line changes visible in the algorithm.

Mapping failure example: topology says “Millennium City Centre Gurugram” while station CSV may use an older or abbreviated spelling. If normalization and alias resolution fail, `mapping.get()` returns `None`; adjacent edges involving the unmatched stop are skipped. This can silently make the destination unreachable. A robust improvement is a startup audit that reports every topology name lacking a canonical match and tests every interchange.

7. Dijkstra in this project

Dijkstra's algorithm finds minimum total non-negative edge weight from one or more origins. It uses a min-heap of `(distance,node)`, repeatedly settles the least-cost unvisited node, and relaxes outgoing edges. MetroInsights starts from every line-specific node of the selected station at distance zero. This correctly treats the starting station's lines as available origins; subsequent line changes require explicit interchange edges. It then folds the line-specific distances back to the minimum per physical station and removes the origin from the result.

Runtime is approximately `O((V+E) log V)` with adjacency lists and a binary heap. Edge values are estimates: coordinates may be placeholders; plausible coordinate distance is converted using a fixed average commercial speed, while missing or implausible data falls back to a default segment duration. Therefore outputs are modeled shortest-path estimates, not timetable-accurate journey times.

8. Technology choices

FastAPI is the HTTP/API framework; Uvicorn is the ASGI server; Jinja2 provides server-rendered views; Pandas/NumPy handle tabular/numeric preparation; scikit-learn implements regression and evaluation; joblib serializes model artifacts; CSV/JSON keep the prototype simple; a custom graph keeps route logic explicit without adding a graph dependency.

9. Limitations and defensible decisions

- Synthetic labels enable controlled experimentation but cannot establish real-world accuracy.
- Historical features reflect generated training data, not an observed operational time series.
- Demand allocation across interchange lines uses a configured split.
- Capacity utilization is against modeled boarding-clearance capacity, not onboard load or platform occupancy.
- Route timings rely on simplified segment and transfer costs.

- APIs accept loosely structured request data; stronger typed validation and unknown-station handling would improve the boundary.
- Static imports load large artifacts and data at process startup.

10. Project pitches

30 seconds: “MetroInsights is a FastAPI-based transit-planning prototype. A user selects a station, date, and time; the app estimates hourly demand with a scikit-learn model trained and evaluated on synthetic scenarios, compares it with configurable service-capacity assumptions, and shows route-time estimates using a line-aware Dijkstra graph. I built the data pipeline, inference flow, service recommendation logic, and dashboard. The key caveat is that the demand data and operational assumptions are synthetic/configurable, so results are planning estimates, not live DMRC measurements.”

1 minute: Add the 23-feature model, chronological holdout and baseline comparisons, calendar-aware controls, minute interpolation, closed-hours handling, and explicit caveats around capacity and routing.

2 minutes: Walk through one request end-to-end, explain offline training vs runtime inference, explain the line-expanded graph and interchange penalties, then acknowledge synthetic validation and name real ridership/timetable data as the path to operational credibility.

5 minutes: Tell it in this sequence: problem/use case; architecture; offline pipeline; API execution; mapping and routing; schedule assumptions; evaluation results; limitations; next steps. Do not lead with metrics before clarifying what the labels represent.

11. Intro interview questions

- Who is the user and what decision does the tool support?
- Which parts are implemented versus simulated?
- What happens from form submission to rendered result?
- Why FastAPI and how is synchronous ML handled?
- Why does the graph use `(station,line)` nodes?
- What do the evaluation numbers mean?
- What is the single biggest validity limitation?
- How would real ridership data change the design?

How to use this series

Read Phase 1 first, then Phase 2 while opening the source files. Use Phases 3, 5, 6, and 7 to deepen technical understanding; rehearse with Phases 4 and 8; revise with Phase 9. The editable Markdown beside each PDF is the source of truth for corrections.

Evidence and honesty rules

MetroInsights is a planning prototype. Its demand targets and holdout are synthetic. Model metrics describe agreement with generated scenario labels, not real passenger counts or operational DMRC accuracy. Network topology, coordinates, capacities, headways, service frequency, and interchange splits are project data or configurable assumptions unless separately verified. Say “estimated” and explain the assumptions. Do not describe this as a live passenger-count, train occupancy, or production DMRC system.

Verified snapshot used in this edition

The cleaned station table contains 243 station records. The topology graph's destination count depends on the selected source and excludes the source itself; 242 destinations can be returned when all other modeled stations are connected. Do not say 226 or 228 without checking the current data. The project defines 11 route keys, including branch keys. The synthetic training series uses 243 stations x 728 dates x 19 hourly slots (05:00-23:00), or 3,361,176 rows. A chronological holdout begins 2026-10-01 and has 438,615 rows. The recorded HistGradientBoostingRegressor scores are MAE 26.58, RMSE 51.10, R2 0.9946, and MAPE 5.18%; Ridge MAE is 53.67 and historical station/day/hour mean MAE is 60.25. These are synthetic-only evaluation results.

Current stack and constraints

FastAPI + Uvicorn, Starlette/Jinja2, server-rendered HTML with JavaScript/CSS, Pandas, NumPy, scikit-learn, joblib, CSV/JSON, and a custom adjacency graph with heapq Dijkstra. No relational database, live ridership feed, authentication, or distributed job queue is present. The generated multi-million-row CSV is a training input; runtime inference loads compact serialized model/metadata plus station and operations inputs, rather than reading every training row on each request. Runtime startup imports routes and eagerly loads artifacts/builds the graph, so missing files can prevent startup. The current FastAPI migration has passed static compilation and diff checks in prior work, but an actual runtime smoke test was not possible in the offline environment; verify locally before claiming deployment verification.